

Transformer 位置编码的前世今生

从绝对位置、相对位置到旋转位置编码 RoPE

Jiguo Li¹

jiguolee@gmail.com

摘要

Self-Attention 能有效建模 token 之间的内容相关性，但其计算本身不包含序列顺序。位置编码的作用，是把绝对坐标、相对距离或旋转相位注入注意力，使模型能够区分词序、表达局部结构，并在更长上下文中保持可用的位置信号。

本文从 Self-Attention 的置换等变性出发，依次推导正弦-余弦位置编码、Shaw 相对位置表示、Transformer-XL、T5 Relative Position Bias 与 RoPE，比较它们注入位置的环节、计算代价、长度外推能力及对 KV Cache 的影响。在此基础上，进一步梳理 Position Interpolation、RoPE scaling law、NTK-aware、Dynamic NTK、NTK-by-parts、YaRN、LongRoPE 和 LongRoPE2，说明这些方法如何调整频率、位置尺度与注意力温度，以及训练长度、目标上下文长度和微调数据之间的约束。全文同时给出公式推导、直观解释、实现要点与一手文献，便于在模型设计和长上下文扩展中据此选择方案。

关键词：Transformer；位置编码；相对位置编码；RoPE；长上下文；长度外推

阅读提要：绝对位置编码为 token 指定坐标；相对位置编码描述 token 对之间的距离；RoPE 则把位置写入 Query/Key 的旋转相位。

¹本文在 Codex 协助下完成

目录

1	为什么 Transformer 必须显式注入位置	3
1.1	RNN、CNN 与 Transformer 的差异	3
1.2	不带位置的 Self-Attention 看到了什么	3
2	技术演进概览	3
3	第一代：绝对位置编码	3
3.1	可学习绝对位置 embedding	4
3.2	正弦-余弦位置编码	4
4	第二代：相对位置编码	5
4.1	Shaw：把相对距离加入 Key 和 Value	5
4.2	Transformer-XL：跨片段记忆中的相对位置	5
4.3	T5：将相对位置压缩为标量 bias	5
5	第三代：旋转位置编码 RoPE	5
5.1	从二维旋转推导核心公式	5
5.2	高维 RoPE 与复数形式	6
5.3	工程实现与配对约定	6
5.4	RoPE 与 KV Cache	6
5.5	RoPE 是否保证距离越远注意力越弱	6
6	RoPE 长上下文扩展：从统一缩放到逐维搜索	7
6.1	问题根源：外推引入未训练相位	7
6.2	RoPE scaling law：基数、训练长度与外推范围	7
6.3	Position Interpolation：统一压缩位置索引	7
6.4	NTK-aware：通过修改 RoPE base 非均匀缩放频率	8
6.5	Dynamic NTK：按当前序列长度动态选择缩放	8
6.6	NTK-by-parts：按频率区间选择插值策略	8
6.7	YaRN：NTK-by-parts 加注意力尺度校准	8
6.8	LongRoPE：逐维、分位置搜索缩放因子	9
6.9	LongRoPE2：从“理论周期”转向“有效训练周期”	9
6.10	方法对比与适用条件	9
7	另一种设计：ALiBi 的距离线性偏置	9
8	三类位置机制的统一视角	10
9	工程实验与评估建议	10
9.1	配置必须随 checkpoint 固化	10
9.2	长上下文评估不能只看“能否输入”	10
10	常见误解	10
11	总结与展望	11
A	代表性 LLM 的位置编码选择	12

1 为什么 Transformer 必须显式注入位置

1.1 RNN、CNN 与 Transformer 的差异

RNN 按时间递推：

$$h_t = f(x_t, h_{t-1}), \quad (1)$$

计算路径本身就是 $x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_n$ ，顺序天然存在于状态传递中。CNN 的卷积核先聚合局部邻域，因此也带有局部结构先验。

Transformer 去掉递归和卷积，让所有 token 并行交互。这是高吞吐的来源，也意味着模型结构本身不再知道 token 的先后顺序。原始 Transformer 因此必须额外注入位置信息 [1]。

1.2 不带位置的 Self-Attention 看到了什么

给定输入矩阵：

$$X = [x_1, x_2, \dots, x_n]^\top \in \mathbb{R}^{n \times d}, \quad (2)$$

标准 Self-Attention 计算：

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad (3)$$

$$\text{Attn}(X) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right)V. \quad (4)$$

第 i 个 Query 对第 j 个 Key 的分数为：

$$s_{ij} = \frac{q_i^\top k_j}{\sqrt{d_h}}. \quad (5)$$

该分数使用了内容向量，却没有直接使用位置 i, j 。设 P 为任意置换矩阵，则：

$$\text{Attn}(PX) = P \text{Attn}(X). \quad (6)$$

因此，不含位置机制的 Self-Attention 是置换等变的：打乱输入后，输出只按相同方式打乱。位置可以在输入表示、注意力分数或 Query/Key 的几何变换中注入，分别对应绝对位置、相对位置和 RoPE 三类方案。

2 技术演进概览

时间	代表工作	核心位置机制
2017	Transformer	固定正弦-余弦绝对位置编码
2018	BERT	可学习绝对位置 embedding
2018	Shaw et al.	在注意力中加入相对距离表示
2019	Transformer-XL	相对位置分解，支持片段级记忆
2020	T5	每个注意力头学习分桶距离 bias
2021	RoFormer / RoPE	按位置旋转 Query 和 Key
2021	ALiBi	对 logits 加线性距离惩罚
2023	PI / YaRN	RoPE 插值、分频缩放与注意力校准
2024–2025	LongRoPE / LongRoPE2	搜索逐维缩放因子并混合长短上下文训练

表 1: Transformer 位置机制的代表性演进。

这些方法不是简单的新旧替代。固定长度 encoder、encoder-decoder 与自回归 LLM 的任务结构、推理方式和上下文需求不同，因此多种方案至今并存。

3 第一代：绝对位置编码

绝对位置编码为位置 i 提供向量 p_i ：

$$h_i^{(0)} = e(x_i) + p_i. \quad (7)$$

它相当于给每个 token 贴上“第 137 位”这样的全局坐标。

3.1 可学习绝对位置 embedding

最直接的实现是维护参数表：

$$P \in \mathbb{R}^{L_{\max} \times d}. \quad (8)$$

第 i 行就是位置 i 的向量 p_i 。BERT 将 token、segment 与 position embedding 相加作为输入 [2]。它实现简单、可直接拟合任务内位置模式，但参数表绑定最大长度；未训练位置没有可靠表示；相邻位置差不被结构性约束；内容与位置在第一层前即混合：

$$q_i = (e(x_i) + p_i)W_Q = e(x_i)W_Q + p_iW_Q. \quad (9)$$

3.2 正弦-余弦位置编码

原始 Transformer 用多组正弦和余弦函数生成位置向量 [1]：

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad (10)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right). \quad (11)$$

定义第 i 组角频率：

$$\theta_i = 10000^{-2i/d_{\text{model}}}, \quad (12)$$

则一组二维表示为：

$$PE_i(pos) = \begin{bmatrix} \sin(pos\theta_i) \\ \cos(pos\theta_i) \end{bmatrix}. \quad (13)$$

完整位置向量由多组不同频率拼接：

$$PE(pos) = [\sin(pos\theta_0), \cos(pos\theta_0), \dots, \sin(pos\theta_{d/2-1}), \cos(pos\theta_{d/2-1})]. \quad (14)$$

3.2.1 “多只钟表”的直观解释

每对维度可看成单位圆上的一根指针，其相位与波长分别为：

$$\phi_i(pos) = pos\theta_i, \quad (15)$$

$$\lambda_i = \frac{2\pi}{\theta_i} = 2\pi \cdot 10000^{2i/d_{\text{model}}}. \quad (16)$$

大的 θ_i 对应“快钟”，能区分相邻位置；小的 θ_i 对应“慢钟”，描述较长尺度。单只钟周期重复，多只不同转速的钟联合后能在更大范围区分位置。常数 10000 是频率基数，不是数学上唯一正确的选择。

当 $d_{\text{model}} = 8$ 时，四组频率为：

$$\theta_0 = 1, \quad \theta_1 = 0.1, \quad \theta_2 = 0.01, \quad \theta_3 = 0.001. \quad (17)$$

3.2.2 为什么必须同时使用 sin 和 cos

由和角公式：

$$\sin((pos + \Delta)\theta) = \sin(pos\theta)\cos(\Delta\theta) + \cos(pos\theta)\sin(\Delta\theta). \quad (18)$$

只保存 sin 无法通过固定线性变换得到平移结果；成对保存后有：

$$PE_i(pos + \Delta) = \begin{bmatrix} \cos(\Delta\theta_i) & \sin(\Delta\theta_i) \\ -\sin(\Delta\theta_i) & \cos(\Delta\theta_i) \end{bmatrix} PE_i(pos). \quad (19)$$

矩阵只依赖位移 Δ 。同一频率下两个位置编码的点积为：

$$PE_i(m)^\top PE_i(n) = \cos((m-n)\theta_i), \quad (20)$$

完整编码满足：

$$PE(m)^\top PE(n) = \sum_i \cos((m-n)\theta_i). \quad (21)$$

所以正弦编码形式上是绝对位置表示，几何关系中却包含相对位移。原始 Transformer 实际使用式 (7)，并未直接以式 (21) 作为注意力分数，模型仍需训练才能利用这种结构。

需要注意：位置函数可以计算到任意 pos ，不等于整个模型能可靠理解任意长上下文。训练长度仍约束注意力分布、状态表示与优化结果。

4 第二代：相对位置编码

语言和代码中的许多关系更关心 $j - i$ ，而不是分别知道 i 与 j 。例如“左侧最近的定义”“距调用点最近的声明”“右侧两个 token 的参数”。

4.1 Shaw：把相对距离加入 Key 和 Value

Shaw 等人将 token 对的相对距离直接加入 Self-Attention[3]。先裁剪距离：

$$r_{ij} = \text{clip}(j - i, -K, K), \quad (22)$$

为每个距离学习相对 Key 向量 $a_{r_{ij}}^K$ ：

$$s_{ij} = \frac{q_i^\top (k_j + a_{r_{ij}}^K)}{\sqrt{d_h}} = \frac{q_i^\top k_j + q_i^\top a_{r_{ij}}^K}{\sqrt{d_h}}. \quad (23)$$

第一项表示内容匹配，第二项表示当前 Query 对某个方向与距离的偏好。Value 也可加入相对表示：

$$z_i = \sum_j \alpha_{ij} (v_j + a_{r_{ij}}^V). \quad (24)$$

4.2 Transformer-XL：跨片段记忆中的相对位置

Transformer-XL 允许当前片段复用上一片段隐藏状态[4]。若缓存携带旧片段的绝对坐标，复用到新片段会产生歧义；相对位置只依赖当前 Query 与历史 Key 的真实距离。其未归一化注意力可分解为：

$$A_{ij} = q_i^\top k_j + q_i^\top W_{k,R} R_{i-j} + u^\top k_j + v^\top W_{k,R} R_{i-j}, \quad (25)$$

分别对应内容-内容、内容-位置、全局内容偏置和全局位置偏置。

4.3 T5：将相对位置压缩为标量 bias

T5 为每个注意力头和距离桶学习一个标量[5]：

$$s_{ij}^{(h)} = \frac{q_i^{(h)\top} k_j^{(h)}}{\sqrt{d_h}} + b_{h, \text{bucket}(j-i)}. \quad (26)$$

距离较小时使用细粒度桶，距离较大时近似对数分桶。不同 head 可学习不同距离先验，形成“内容相似度 + 距离偏好”的清晰分解。

传统相对位置方法在建模上直观，但通常要处理 $n \times n$ 个 token 对。相对信息可能进入 Key、Value、logits 或多个交叉项，增加算子融合、增量解码与高效注意力实现的复杂度。RoPE 的关键价值，就是在显式表达相对位移的同时保留标准 $QK^\top$ 形式。

5 第三代：旋转位置编码 RoPE

RoPE 不把位置向量加到输入，也不为 token 对显式构造相对向量，而是根据绝对位置旋转 Query 和 Key[6]。

5.1 从二维旋转推导核心公式

二维旋转矩阵为：

$$R(\phi) = \begin{bmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{bmatrix}. \quad (27)$$

位置 m 的旋转角为 $m\theta$ ：

$$q'_m = R(m\theta)q_m, \quad k'_n = R(n\theta)k_n. \quad (28)$$

旋转后的点积为：

$$\begin{aligned} q_m'^\top k'_n &= q_m^\top R(m\theta)^\top R(n\theta)k_n \\ &= q_m^\top R((n-m)\theta)k_n. \end{aligned} \quad (29)$$

这里使用了：

$$R(m\theta)^\top = R(-m\theta), \quad R(-m\theta)R(n\theta) = R((n-m)\theta). \quad (30)$$

单个向量 q'_m 依赖绝对位置 m ；Query-Key 交互只通过 $n-m$ 依赖相对位置。因此，**RoPE** 在单个向量上编码绝对位置，在点积中呈现相对位置。

5.2 高维 RoPE 与复数形式

将 head dimension 两两组成二维子空间，第 r 对频率为：

$$\theta_r = b^{-2r/d_h}, \quad r = 0, 1, \dots, d_h/2 - 1, \quad (31)$$

经典基数 $b = 10000$ 。整体旋转矩阵为：

$$R_m = \text{diag}(R(m\theta_0), R(m\theta_1), \dots, R(m\theta_{d_h/2-1})). \quad (32)$$

若把二维向量写成复数 $z_r = x_{2r} + ix_{2r+1}$ ，RoPE 等价于：

$$z'_r = z_r e^{im\theta_r}. \quad (33)$$

Query 与 Key 的相位因子相乘后为：

$$e^{-im\theta_r} e^{in\theta_r} = e^{i(n-m)\theta_r}, \quad (34)$$

即绝对相位相减后只剩相对相位。

5.3 工程实现与配对约定

以下实现使用相邻维度配对。某些框架采用前半维与后半维配对；两者只是排列约定不同，但权重、cos/sin cache 与旋转函数必须一致。

```
def apply_rope(x, cos, sin):
    x_even = x[..., 0::2]
    x_odd = x[..., 1::2]
    y_even = x_even * cos - x_odd * sin
    y_odd = x_even * sin + x_odd * cos

    y = torch.empty_like(x)
    y[..., 0::2] = y_even
    y[..., 1::2] = y_odd
    return y
```

Value 通常不旋转，因为位置主要影响“关注谁”的权重；聚合对象仍是内容 Value。

5.4 RoPE 与 KV Cache

固定 RoPE 配置下，历史 Key 通常以旋转后的形式缓存：

$$k'_i = R_i k_i. \quad (35)$$

生成位置 t 时只需计算：

$$q'_t = R_t q_t, \quad k'_t = R_t k_t. \quad (36)$$

历史 Key 无需重算。工程上最易出错的是 position offset：prefill、decode、padding、sequence packing、prefix cache 与 sliding window 必须对同一 token 使用一致的逻辑 position id。

5.5 RoPE 是否保证距离越远注意力越弱

单个频率的相对项包含：

$$\cos((n-m)\theta_r), \quad \sin((n-m)\theta_r), \quad (37)$$

它们随距离周期振荡，并非单调衰减。RoFormer 讨论了多频率汇总后的远距离衰减倾向 [6]，但不能对任意固定 Query/Key、任意 head 和任意距离推出“越远分数必然越小”。

6 RoPE 长上下文扩展：从统一缩放到逐维搜索

6.1 问题根源：外推引入未训练相位

设原始训练长度为 L_0 ，目标长度为 $L_1 = sL_0$ ，其中 $s > 1$ 。第 r 个二维子空间的周期为：

$$T_r = \frac{2\pi}{\theta_r} = 2\pi b^{2r/d_h}. \quad (38)$$

高频维度在 L_0 内经历许多周期，局部模式训练充分；低频维度可能连一个完整周期都未覆盖。直接推理到 L_1 会让部分维度进入未见过的相位区间，造成 RoPE OOD。LongRoPE2 将理论临界维度定义为周期是否超过原训练长度的分界 [12]：

$$r_{\text{crit}} = \min\{r : T_r \geq L_0\}. \quad (39)$$

扩展方法真正需要处理的是各频率的缩放方式，以及模型权重对新相位分布的适应，而不仅是增大 position id 的取值范围。

6.2 RoPE scaling law：基数、训练长度与外推范围

Liu 等人从周期覆盖的角度研究了 RoPE 的长度外推，并提出 RoPE-based extrapolation scaling laws[9]。这里的“scaling law”描述的是旋转基数、训练长度与外推范围之间的关系，与参数量、数据量和计算量之间的经典模型 scaling law 含义不同。

对原始基数 10000，第 n 组频率的周期为：

$$T_n = \frac{2\pi}{\theta_n} = 2\pi \cdot 10000^{2n/d_h}. \quad (40)$$

若预训练长度为 T_{train} ，论文把能够在训练区间内至少覆盖完整周期的维度上限写为：

$$d_{\text{extra}} = 2 \left\lceil \frac{d_h}{2} \log_{10000} \left(\frac{T_{\text{train}}}{2\pi} \right) \right\rceil. \quad (41)$$

超过 d_{extra} 的低频维度在训练中没有观察到完整周期，超出训练长度后更容易出现相位分布偏移。以 LLaMA 2 的 $d_h = 128$ 、 $T_{\text{train}} = 4096$ 为例，论文计算得到 $d_{\text{extra}} = 92$ ；其余 36 个维度被视为外推时更容易失稳的部分 [9]。

当微调阶段把旋转基数改为 $\beta > 10000$ ，该工作用临界维度的新周期估计外推上限：

$$T_{\text{extra}} = 2\pi \beta^{d_{\text{extra}}/d_h}. \quad (42)$$

反过来，给定期望长度 $\tilde{T}_{\text{extra}}$ ，可由论文给出的关系估计所需的临界基数：

$$\beta_0 = 10000^{\log_{T_{\text{train}}/(2\pi)}(\tilde{T}_{\text{extra}}/(2\pi))}. \quad (43)$$

论文还观察到，在固定微调长度下，将基数调小也可能改善外推，因为更多维度能在训练区间内经历更充分的三角函数变化。对应的三个相位覆盖点为：

$$\beta_1 = \frac{2T_{\text{train}}}{\pi}, \quad \beta_2 = \frac{T_{\text{train}}}{\pi}, \quad \beta_3 = \frac{T_{\text{train}}}{2\pi}. \quad (44)$$

这组结果揭示了 RoPE 外推与周期覆盖之间的联系，但不宜把式 (42) 当成跨模型的硬性上下文上限。论文主要在 LLaMA 2 上验证，实际有效长度还受到语料中的长距离依赖分布、微调数据、attention head 行为和评测任务影响。

6.3 Position Interpolation：统一压缩位置索引

PI 将目标范围中的位置压回原训练区间 [8]：

$$m' = \frac{m}{s} = m \frac{L_0}{L_1}. \quad (45)$$

等价地，对所有频率统一缩放：

$$\theta'_r = \frac{\theta_r}{s}. \quad (46)$$

其优点是所有新位置都落回已训练相位范围，结构改动极小；缺点是高频也被压缩，相邻 token 的相位差从 θ_r 变成 θ_r/s ，局部分辨率下降。PI 论文从 2K 扩至 32K，仅需短程微调即可适应，但也明确指出原长度内会有一定性能损失 [8]。

6.4 NTK-aware: 通过修改 RoPE base 非均匀缩放频率

NTK-aware 的直觉是：不要像 PI 一样等比例压缩所有频率，而是调整频率基数 b ，使高频尽量保持、低频承担更多扩展。常见形式把新基数设为：

$$b' = b s^{d_h/(d_h-2)}, \quad (47)$$

从而：

$$\theta'_r = (b')^{-2r/d_h} = \theta_r s^{-2r/(d_h-2)}. \quad (48)$$

当 $r = 0$ 时最高频保持不变；随 r 增大，低频被更强地缩放。它在局部精度和长程覆盖之间比统一 PI 更平滑。需要注意，NTK-aware 最初来自开源社区经验，后来由 YaRN 论文系统整理，而不是一开始就有独立同行评议论文 [10]。

6.5 Dynamic NTK: 按当前序列长度动态选择缩放

固定目标长度 L_1 的缩放会使短输入也承受位置压缩。Dynamic Scaling 根据当前序列长度 ℓ 更新倍率 [10]：

$$s(\ell) = \max\left(1, \frac{\ell}{L_0}\right), \quad (49)$$

再将 $s(\ell)$ 代入式 (47)。短序列保持原 RoPE，超过 L_0 后逐步扩展。

这一做法与旋转后 KV Cache 存在冲突：当 $s(\ell)$ 变化时，同一个历史 token 的旋转角也变化。若缓存的是已经旋转的 Key，旧缓存便与新倍率不一致。严格实现要么缓存旋转前 Key 并在倍率变化后重新旋转，要么在一次生成会话内固定倍率；YaRN 论文也明确提醒了这一点 [10]。

6.6 NTK-by-parts: 按频率区间选择插值策略

YaRN 用原训练长度内的旋转圈数衡量第 r 个频率的训练覆盖：

$$\rho(r) = \frac{L_0 \theta_r}{2\pi}. \quad (50)$$

若 $\rho(r)$ 很小，说明该维度在训练区间内尚未完整旋转，应像 PI 一样充分插值以避免 OOD；若 $\rho(r)$ 很大，说明它已经多次旋转，应尽量保留以维持局部分辨率。定义分段线性 ramp：

$$\gamma(\rho) = \begin{cases} 0, & \rho < \alpha, \\ \frac{\rho - \alpha}{\beta - \alpha}, & \alpha \leq \rho \leq \beta, \\ 1, & \rho > \beta, \end{cases} \quad (51)$$

则 NTK-by-parts 的新频率可写为：

$$\theta'_r = (1 - \gamma(\rho(r))) \frac{\theta_r}{s} + \gamma(\rho(r)) \theta_r. \quad (52)$$

低圈数频率接近 PI，高圈数频率保持原值，中间频率平滑过渡。YaRN 对 LLaMA 系列给出的经验设置为 $\alpha = 1, \beta = 32$ ，但这不是跨模型通用常数 [10]。

6.7 YaRN: NTK-by-parts 加注意力尺度校准

扩大后 attention entropy 往往改变。YaRN 在 NTK-by-parts 基础上加入 attention scaling [10]：

$$\alpha_{mn} = \text{softmax}_n \left(\frac{q_m^\top k_n}{t \sqrt{d_h}} \right), \quad (53)$$

并可通过同时缩放 q, k 来等价实现：

$$\tilde{q} = \frac{q}{\sqrt{t}}, \quad \tilde{k} = \frac{k}{\sqrt{t}}. \quad (54)$$

论文对 LLaMA/Llama 2 给出经验关系：

$$\frac{1}{\sqrt{t}} = 0.1 \ln s + 1. \quad (55)$$

因此 YaRN 不是“另一种单一插值公式”，而是分频率插值 + 注意力温度校准。论文报告它用比 PI 少约 $2.5\times$ 的训练步数和数据达到相近扩展效果 [10]。实际框架中的 factor、beta_fast、beta_slow、attention_factor 等命名可能不同，必须与 checkpoint 训练配置一致。

6.8 LongRoPE: 逐维、分位置搜索缩放因子

前述方法都用解析规则生成缩放。LongRoPE 进一步利用两种非均匀性 [11]:

1. 不同 RoPE 维度需要不同缩放因子 λ_r ;
2. 序列开头一段位置对插值更敏感, 可设置保留区间 n_{hat} 。

其一般化频率写作:

$$\theta'_r = \frac{\theta_r}{\lambda_r}, \quad (56)$$

并用 evolutionary search 在约束空间中寻找 $\{\lambda_r\}$ 与 n_{hat} 。LongRoPE 先在 256K 长度适配, 再基于该模型进行第二阶段搜索与插值, 论文报告达到 2048K; 同时为短上下文搜索另一套缩放配置以恢复原长度性能 [11]。关键思想是: 扩展倍率不应被假设为跨维度、跨位置均匀。

6.9 LongRoPE2: 从“理论周期”转向“有效训练周期”

LongRoPE2 指出, 低频/高维 RoPE 在预训练语料中不仅理论周期长, 而且真正发生长距离依赖的样本稀少, 导致其有效相位范围比理论预测更窄 [12]。因此, 理论临界维度未必等于实际临界维度。它做了三项改进:

1. 使用 needle-driven perplexity 指导 evolutionary search, 联合寻找实际临界维度与逐维缩放因子;
2. 对 OOD 维度施加单调非减的 λ_r 约束;
3. 混合上下文训练: 短样本使用原始 RoPE, 长样本使用 rescaled RoPE。

混合训练目标可抽象为:

$$\mathcal{L} = \eta \mathcal{L}_{\text{short}}(\theta_{\text{orig}}) + (1 - \eta) \mathcal{L}_{\text{long}}(\theta_{\text{scaled}}), \quad (57)$$

它直接处理“长上下文适配”和“短上下文保持”之间的冲突。论文在 LLaMA3-8B 与 Phi3-mini 上报告 128K 有效上下文与较高短上下文保持率; 这些数字是论文实验结论, 不应外推为所有模型的默认保证 [12]。

6.10 方法对比与适用条件

方法	缩放对象	主要优点	主要代价/风险
PI	所有位置/频率统一除以 s	稳定、实现最简单	高频局部分辨率下降, 通常需微调
NTK-aware	修改 base, 逐维非均匀缩放	保留最高频局部结构	社区经验起源, 超参依模型
Dynamic NTK	倍率随当前长度变化	短输入保持原 RoPE	与旋转后 KV Cache 冲突
NTK-by-parts	按旋转圈数分区插值	高频保留、低频防 OOD	需选择 α, β
YaRN	NTK-by-parts + 温度校准	长短期折中更完整	必须匹配训练 recipe 与 attention factor
LongRoPE	搜索逐维因子与位置保留区	能利用非均匀性, 支持渐进扩长	搜索与验证成本更高
LongRoPE2	搜索有效临界维度 + 混合训练	同时优化有效长度和短程保持	需要专门数据、搜索和 mid-training

表 2: 主要 RoPE 扩展方法的统一比较。

实践建议: 对已有 checkpoint, 应复用其原生 RoPE 配置和训练 recipe; rope_theta、PI factor、YaRN factor 与 LongRoPE 的逐维因子并不等价。从头训练长上下文模型时, 需要联合设计 RoPE scaling、长短样本 mix、position-id 语义和评估矩阵。

7 另一种设计：ALiBi 的距离线性偏置

ALiBi 不旋转向量, 而是在第 h 个 head 的注意力分数中加入线性距离惩罚 [7]:

$$s_{ij}^{(h)} = \frac{q_i^{(h)\top} k_j^{(h)}}{\sqrt{d_h}} - m_h |i - j|. \quad (58)$$

不同 head 使用不同斜率 m_h 。它形式简单、没有位置表, 并显式注入 recency bias; 但其归纳偏置与 RoPE 的多频率相位不同, 是相对位置 bias 分支而非 RoPE 扩展。

8 三类位置机制的统一视角

三类方法分别修改输入、token 对分数和 Query-Key 几何关系：

$$x'_i = x_i + p_i, \quad \text{绝对位置,} \quad (59)$$

$$s_{ij} = \frac{q_i^\top k_j}{\sqrt{d_h}} + b(i - j), \quad \text{相对位置 bias,} \quad (60)$$

$$s_{ij} = \frac{(R_i q_i)^\top (R_j k_j)}{\sqrt{d_h}} = \frac{q_i^\top R_{j-i} k_j}{\sqrt{d_h}}, \quad \text{RoPE.} \quad (61)$$

机制	注入位置	相对距离	超长位置可计算	可靠外推条件
Learned Absolute	输入	不显式	通常否	受位置表与训练长度限制
Sinusoidal	输入	几何中隐含	是	可计算不等于可利用
Relative Bias	logits/K/V	显式	依分桶而定	取决于分桶与训练
RoPE	Q/K	点积中显式	是	通常需 scaling 与适配训练
ALiBi	logits	显式	是	具有强 recency bias

表 3: 位置机制的结构比较。

9 工程实验与评估建议

9.1 配置必须随 checkpoint 固化

对 decoder-only LLM，至少应保存并验证：

- rope_theta 或 base frequency；
- rotary dimension：全维或 partial rotary；
- 维度配对布局；
- 原始训练长度与目标长度；
- scaling 类型、factor、attention factor 与逐维参数；
- packing、prefix cache、sliding window 下 position id 的定义。

9.2 长上下文评估不能只看“能否输入”

建议至少报告：

1. 短上下文保持率：扩长前后原长度 PPL 与 benchmark；
2. 按 token position、依赖跨度分桶的 PPL；
3. needle/passkey 的长度-深度二维网格；
4. RULER 类多任务长上下文评估，而非单一检索；
5. 有效上下文：性能开始显著退化的位置，而非配置最大长度；
6. prefill/decode latency、峰值显存、KV Cache 占用与吞吐。

代码模型还应补充 definition-use 跨度、跨文件 import/call graph 定位、同名符号消歧、repo-level completion 与 patch correctness，并验证扩长后 HumanEval 类短程语法和算法能力是否退化。

10 常见误解

1. **Self-Attention** 完全看不见顺序。更准确地说，不含位置机制的 Self-Attention 对输入置换等变。causal mask 提供可见方向，但不等于精细距离表示。
2. 正弦编码是绝对位置，所以不含相对信息。式 (19) 与式 (20) 表明固定偏移对应固定旋转，点积只依赖 $m - n$ 。
3. **RoPE** 是相对位置编码，因此不编码绝对位置。单个 $R_m q_m$ 明确依赖 m ；只有点积时绝对相位才相消。
4. 位置函数能算到 **128K**，模型就有 **128K** 上下文。配置长度、数值可运行长度与有效上下文长度是三件事。
5. **RoPE** 天然让注意力随距离单调下降。式 (37) 是周期函数，不存在逐样本严格单调保证。

6. 只调大 `rope_theta` 就完成扩长。频率缩放只是第一步，还要验证模型适配、KV Cache 一致性、短程保持与有效上下文。

11 总结与展望

绝对位置编码把坐标写入 token 表示，相对位置编码直接修改 token 对之间的关系，RoPE 则通过 Query-Key 旋转把绝对位置转化为点积中的相对相位。PI、NTK-aware、YaRN 和 LongRoPE 进一步处理频率缩放与长短上下文兼容问题；RoPE scaling law 则从周期覆盖出发，联系 base、训练长度、临界维度与外推范围。不过，现有位置机制仍有若干问题没有解决：

1. 标称长度与有效长度不一致。能接收 128K 或 1M token，不代表能稳定利用所有位置；有效上下文还受训练依赖跨度、优化过程和注意力稀释影响。
2. 周期性带来相位混叠。不同距离可能产生相近相位；多频率组合能够缓解，却不能从理论上消除这一问题。
3. 扩长与短程能力仍有冲突。插值会降低局部分辨率，逐维缩放和混合长度训练也缺少可跨模型迁移的稳定规律。
4. 系统实现可能破坏位置语义。packing、prefix cache、sliding window、speculative decoding 和 KV Cache 复用都可能造成隐蔽的 position-id 错位。
5. 一维距离难以表达复杂结构。代码的文件层级、调用图和 definition-use 关系，以及多模态数据的空间与时间结构，都超出单一序列坐标的表达范围。
6. 位置编码与高效注意力缺少统一设计。稀疏注意力、滑窗、KV 压缩和线性注意力会改变信息传播路径，位置机制需要与算子共同设计。
7. 评估方法仍然偏弱。needle-in-a-haystack 不能充分反映多跳推理、跨文件代码理解和长程因果依赖。

后续研究不应只放大标称上下文，而应建立可预测、可训练、可缓存且能表达多维结构的位置机制，并用真实长依赖任务检验远距离信息利用率。对代码模型而言，definition-use、跨文件调用和 repository-level 修改比随机 needle 更接近实际需求。

A 代表性 LLM 的位置编码选择

表 4 汇总公开论文、技术报告或官方开源配置中可以核验的位置机制，资料状态更新至 2026 年 8 月 7 日。表中的“上下文扩展”只记录位置编码相关做法，不代表模型仅凭该机制就能达到标称上下文长度。对于没有公开架构细节的模型，不能依据上下文长度或同系列旧模型反推出其位置编码。

表 4: 代表性语言模型公开披露的位置编码方式。

模型/系列	位置机制	技术报告中的相关设计
GPT-2 / GPT-3	可学习绝对位置 embedding	GPT-2 使用可学习的位置 embedding; GPT-3 沿用 GPT-2 的基本 Transformer 架构, 因此仍采用 learned absolute 方案 [13, 14]。这类设计的最大长度通常受位置表和训练长度约束。
T5	分桶相对位置 bias	每个 attention head 对相对距离桶学习标量偏置; 近距离细分、远距离近似对数分桶 [5]。
BLOOM	ALiBi	不向 embedding 加位置向量, 而是在注意力 logits 上施加按距离增长的线性负偏置 [15]。
Falcon	ALiBi	Falcon 系列技术报告采用 ALiBi; 其消融实验同时比较了 ALiBi、URPE 与 RoPE [16]。
PaLM	RoPE	使用 RoPE 取代绝对或相对位置 embedding; 报告将其长序列表现列为采用理由之一 [17]。
LLaMA / Llama 2	RoPE	LLaMA 以 RoPE 取代绝对位置 embedding; Llama 2 延续 RoPE, 并把预训练上下文从 2K 扩为 4K [18, 19]。
Code Llama	RoPE, 增大基数	在 Llama 2 基础上把 RoPE base 调为 10^6 , 使用 16K 序列继续训练; 报告研究了向 100K 的外推 [20]。
Llama 3	RoPE, $\theta = 500,000$	技术报告把 RoPE base 提高到 500K, 用于改善长上下文支持; 该配置对 8B、70B 与 405B 模型一致 [21]。
Qwen2	RoPE + YaRN + DCA	预训练后期把上下文从 4K 扩到 32K, 并把 RoPE base 从 10K 调为 10^6 ; 推理侧结合 YaRN 与 Dual Chunk Attention 支持更长输入 [22]。
Qwen3 / Qwen3.6	RoPE 系列 + YaRN	Qwen3 的官方配置使用 $\theta = 10^6$, 可通过 YaRN 从原生 32K 扩展到 131K [27]。Qwen3.6 进一步采用全注意力与线性注意力混合结构; 其官方配置在全注意力层使用部分旋转的 M-RoPE, $\theta = 10^7$, 原生支持 262K, 并给出使用 YaRN 扩展至约 1M 的配置 [28]。
Gemma	RoPE	Gemma 在每层使用 rotary positional embeddings, 而不是绝对位置 embedding; 基础模型训练长度为 8K [23]。
Phi-3 / Phi-3.5	RoPE 系架构 + LongRoPE	Phi-3-mini 采用与 Llama 2 相近的 block; 128K 版本通过 LongRoPE 扩展, Phi-3.5 继续使用 LongRoPE 与混合上下文训练 [24]。
DeepSeek-V2 / V3	MLA 中的 decoupled RoPE	为兼容低秩 KV 压缩, 将携带 RoPE 的 Query/Key 子空间与内容压缩子空间解耦; V3 延续该 MLA 设计 [25, 26]。
LongCat 系列	MLA 中的 decoupled RoPE	LongCat-Flash、LongCat-Flash-Thinking 与 2601 版本沿用 MLA, 把 Query/Key 分成 RoPE 与 NoPE 子空间。初版 Flash 的官方配置为 64 维 RoPE 子空间、128 维 NoPE 子空间, $\theta = 10^7$, 最大位置为 131K; 2601 的公开配置将 θ 设为 10^6 [29, 30, 31]。
Kimi K3	NoPE + Kimi Delta Attention	不向 MLA 的 Query/Key 注入显式位置编码; 位置敏感性由 KDA 的递归门控与衰减机制隐式提供。技术报告指出该设计无需 RoPE rescaling 或 interpolation 即可继续训练到 1M 上下文 [32]。

下页续

表 4 (续)

模型/系列	位置机制	技术报告中的相关设计
GLM-5	MLA/DSA 中的 decoupled RoPE	GLM-5 以 DSA 稀疏化 MLA 的注意力计算；官方配置保留 64 维 RoPE 子空间与 192 维 NoPE 子空间，并设置 $\theta = 10^6$ 。DSA 改变稀疏检索路径，但没有取消 RoPE[33, 34]。
Qwen3.8-Max	未公开	官方服务文档已列出 Qwen3.8-Max，但截至本文资料截止日尚无公开技术报告、权重或配置可以确认位置机制。因此不能直接套用 Qwen3/Qwen3.6 的 RoPE 方案 [35]。
GPT-4	未公开	GPT-4 技术报告未披露模型结构、位置编码或 RoPE 配置，因此无法从公开材料确定其位置机制 [36]。

从公开资料可以看到，decoder-only 开源模型在 2022 年以后明显集中到 RoPE，但并未完全收敛：BLOOM 与 Falcon 使用 ALiBi，T5 系模型采用相对位置 bias，DeepSeek、LongCat 与 GLM-5 为适配 MLA 或稀疏注意力而拆分 RoPE/NoPE 子空间，Kimi K3 则转向由 KDA 隐式提供位置信息的 NoPE 方案。所谓“使用 RoPE”也不是一个完整配置，base、rotary dimension、scaling、训练长度和 position-id 语义都会改变实际行为；对于 Qwen3.8-Max 这类未披露架构的模型，表中保留“未知”比沿用系列历史配置更可靠。

参考文献

- [1] A. Vaswani et al. *Attention Is All You Need*. NeurIPS, 2017.
- [2] J. Devlin et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL, 2019.
- [3] P. Shaw, J. Uszkoreit, A. Vaswani. *Self-Attention with Relative Position Representations*. NAACL, 2018.
- [4] Z. Dai et al. *Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context*. ACL, 2019.
- [5] C. Raffel et al. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. JMLR, 2020.
- [6] J. Su et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. 2021.
- [7] O. Press, N. A. Smith, M. Lewis. *Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation*. ICLR, 2022.
- [8] S. Chen et al. *Extending Context Window of Large Language Models via Position Interpolation*. 2023.
- [9] X. Liu et al. *Scaling Laws of RoPE-based Extrapolation*. ICLR, 2024.
- [10] B. Peng et al. *YaRN: Efficient Context Window Extension of Large Language Models*. ICLR, 2024; arXiv revised 2026.
- [11] Y. Ding et al. *LongRoPE: Extending LLM Context Window Beyond 2 Million Tokens*. ICML, 2024.
- [12] N. Shang et al. *LongRoPE2: Near-Lossless LLM Context Window Scaling*. 2025.
- [13] A. Radford et al. *Language Models are Unsupervised Multitask Learners*. OpenAI, 2019.
- [14] T. Brown et al. *Language Models are Few-Shot Learners*. NeurIPS, 2020.
- [15] T. Le Scao et al. *BLOOM: A 176B-Parameter Open-Access Multilingual Language Model*. 2022.
- [16] E. Almazrouei et al. *The Falcon Series of Open Language Models*. 2023.
- [17] A. Chowdhery et al. *PaLM: Scaling Language Modeling with Pathways*. 2022.
- [18] H. Touvron et al. *LLaMA: Open and Efficient Foundation Language Models*. 2023.
- [19] H. Touvron et al. *Llama 2: Open Foundation and Fine-Tuned Chat Models*. 2023.
- [20] B. Rozière et al. *Code Llama: Open Foundation Models for Code*. 2023.
- [21] A. Dubey et al. *The Llama 3 Herd of Models*. 2024.
- [22] A. Yang et al. *Qwen2 Technical Report*. 2024.
- [23] Gemma Team. *Gemma: Open Models Based on Gemini Research and Technology*. 2024.
- [24] M. Abdin et al. *Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone*. 2024.
- [25] DeepSeek-AI. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. 2024.
- [26] DeepSeek-AI. *DeepSeek-V3 Technical Report*. 2024.
- [27] Qwen Team. *Qwen3 Technical Report*. 2025.
- [28] Qwen Team. *Qwen3.6-35B-A3B Official Model Configuration; Processing Ultra-Long Texts*. 2026.
- [29] Meituan LongCat Team. *LongCat-Flash Technical Report*. 2025.
- [30] Meituan LongCat Team. *LongCat-Flash-Thinking-2601 Technical Report*. 2026.
- [31] Meituan LongCat Team. *LongCat-Flash-Chat Official Model Configuration; LongCat-Flash-Thinking-2601 Official Model Configuration*. 2025–2026.
- [32] Kimi Team. *Kimi K3: Open Frontier Intelligence*. 2026.
- [33] GLM-5 Team. *GLM-5: From Vibe Coding to Agentic Engineering*. 2026.
- [34] Z.ai. *GLM-5 Official Model Configuration*. 2026.
- [35] Alibaba Cloud. *Model Studio: Supported Models*. Accessed 2026-08-07.
- [36] OpenAI. *GPT-4 Technical Report*. 2023.